Jenkins CI/CD

Practice Lab

Bookshelf API — Freestyle Job with Python & Flask

1. Overview

In the class session you automated a static website pipeline end to end. Now it is your turn to fly solo.

In this lab you will build a complete CI/CD pipeline for a **Python Flask REST API** called the ***Bookshelf API***. It is a different tech stack from what you saw in class — intentionally. Real DevOps engineers wire pipelines to applications they did not write. Your job is not to code the app; your job is to **automate it**.

> **End Goal:** Push a commit → Jenkins detects the change → tests run → artifact is packaged → you are notified. Zero manual clicks after the initial setup.

What this lab covers

- Creating and configuring a Freestyle Job
- Connecting Jenkins to a GitHub repository via SCM
- Comparing Poll SCM vs. Webhook triggers
- Installing Jenkins plugins
- Writing a multi-step shell build script
- Archiving build artifacts
- Configuring post-build notifications
- Reading Console Output to debug pipeline failures

Environment

- Jenkins is already running and accessible.
- Log in with the credentials provided **(admin/admin).**
- The application repository is at: `https://github.com/scaler2101/jenkins-bookshelf`

2. The Application — Bookshelf API

Before touching Jenkins, spend a few minutes understanding what you are automating. Clone the repository and explore it.

```
git clone https://github.com/scaler2101/jenkins-bookshelf.git
cd jenkins-bookshelf
```

Repository Structure

```
jenkins-bookshelf/
├── app/
│   └── app.py          ← Flask application (the source code)
├── tests/
│   └── test_app.py     ← Automated test suite (pytest)
├── requirements.txt    ← Python dependencies
└── README.md
```

API Endpoints

Method	Path	Description
GET	/	Service info (name, version, status)
GET	/health	Health check — returns {status: healthy}
GET	/books	List all books
GET	/books?available=true	List only available books
GET	/books/<id>	Get a specific book by ID
POST	/books	Add a new book (JSON body required)

Run the App Locally First

Always verify an application works before trying to automate it. This separates application bugs from pipeline bugs.

> **Note:** Commands below use `python` and `pip`. If those are not found on your system, try `python3` and `pip3` instead. Run `python --version` or `python3 --version` first to see what is available.

```
# Install dependencies
pip install -r requirements.txt

# Start the app
python app/app.py
```

```
# In a second terminal — verify it responds
curl http://localhost:5000/health
# Expected response: {"status": "healthy"}

# Try another endpoint
curl http://localhost:5000/books
```

Run the Tests Locally

The test suite uses pytest. Run it once so you know what a passing run looks like before Jenkins does it for you.

```
# Try this first
pytest tests/ -v

# If pytest command is not found, use:
python -m pytest tests/ -v

# Or with python3:
python3 -m pytest tests/ -v

# All 10 tests should pass
```

Note: `pytest` may not be on your PATH even after installing it — this is common on macOS. Using `python -m pytest` or `python3 -m pytest` invokes it directly through Python and sidesteps that issue entirely.
The --cov flag adds coverage reporting. You can run: pytest tests/ -v --cov=app --cov-report=term-missing — this shows which lines of app.py are (and are not) covered by the tests.

Ensure pytest is installed and the pytest binary location is in your PATH

3. Tasks

Each task below describes ***what*** you need to achieve and ***why*** it matters. How you get there is up to you — that is the point of this lab. Use Jenkins documentation, the class notes, and your own judgment.

Task 1: Install the Required Plugins

Jenkins ships with a core feature set. Some capabilities require additional plugins.

Plugins to install: Slack Notification, Workspace Cleanup

Find them in the Plugin Manager and install it. No restart should be needed.

Think about: How do plugins follow the idea of keeping Jenkins lean — only adding what you need?

Task 2: Create the Freestyle Job

Create a new Jenkins job with the following properties:

- Job name: `Bookshelf-API-Pipeline`
- Job type: Freestyle project

Think about: What is the difference between a Freestyle project and a Pipeline project in Jenkins? When would you choose one over the other?

Task 3: Connect Jenkins to GitHub (SCM)

Configure your job to pull code from your fork of this repo:
`https://github.com/scaler2101/jenkins-bookshelf`

- The job must clone the correct branch (main).
- No credentials are required — the repository is public.

> Pay attention to the Branch Specifier field. The default value Jenkins fills in may not match the actual branch name in this repository.

Think about: What happens if you leave the Branch Specifier set to */master when the repo's default branch is main?

Task 4: Configure a Build Trigger

Jenkins needs to know when to run a build. Configure Poll SCM to check the repository for new commits on a regular schedule.

- Enable **Poll SCM** as the build trigger.
- Set the schedule so Jenkins polls **every minute**.

The cron syntax Jenkins uses has 5 fields: `MINUTE HOUR DOM MONTH DOW`. A `*` in a field means 'every'. Figure out the right combination.

> Once you have Poll SCM working, read about the **GitHub hook trigger for GITScm polling** option. Understand how a webhook differs from polling — why is it more efficient? *Implementing the webhook is a stretch goal* covered in Task 7.

Think about: Poll SCM uses cron to repeatedly ask GitHub 'anything new?'. What is the downside of this approach at scale, with dozens of jobs polling every minute?

Task 5: Configure the Build Environment

Before every build, the workspace should be in a clean state — no leftover files from previous builds.

- Enable the option that deletes the workspace before the build starts.

Think about: If you did not clean the workspace, and build #3 produced a file that build #4 should have produced but failed to — what would the test result look like? Would it be a true result?

Task 6: Write the Build Steps (Execute Shell)

This is the core of the pipeline. Add an **Execute shell** build step. Your script must accomplish the following **four things, in order**:

#	Step	What is expected
1	**Install dependencies**	Install all packages listed in requirements.txt
2	**Run the test suite**	Run pytest against the tests/ folder. All 10 tests must pass. If any test fails, the build must fail and stop here.
3	**Package the artifact**	Create a zip file named bookshelf-api-build-<BUILD_NUMBER>.zip containing the app/ folder and requirements.txt. (Jenkins provides the build number as an environment variable.)

4	**Verify the artifact**	Confirm the zip file actually exists. If it does not, fail the build explicitly.

Python / pip note: In your shell script, try `python` and `pip` first. If the command is not found, use `python3` and `pip3` instead. You can check what is available by running `which python python3` in the Jenkins Execute Shell box.

Hint: Look into the `set -e` bash directive. It controls what happens when a command in your script exits with a non-zero code. Think about where you want the pipeline to stop if something goes wrong.

Think about: Jenkins exposes build metadata as environment variables. `${BUILD_NUMBER}` is one of them. Why is including the build number in the artifact name useful in a real deployment workflow?

Task 7: Archive the Build Artifact

After a successful build, the zip file must be stored by Jenkins so it can be downloaded from the build page.

- Add a **post-build action** to archive artifacts.
- The file pattern must match: `bookshelf-api-build-*.zip`

> The wildcard `*` in the pattern is essential because the filename changes with every build (it includes the build number). A fixed filename pattern would fail to find the artifact.

Think about: What is the difference between a build artifact stored in Jenkins and a deployment? At what point would you take this artifact and push it to a server or container registry?

Task 8: Configure Slack Notifications

Post-build actions run after the main build steps — whether the build passed or failed. Configure Jenkins to send a Slack message on both outcomes.

- Add the **Slack Notifications** post-build action.
- Enable notifications for both **Success** and **Failure**.
- Configure the Slack workspace and token in Manage Jenkins → System (your instructor will provide credentials, or this step may be skipped depending on lab scope).

Think about: Why is a failure notification arguably more important than a success notification in a team environment? Who should receive it, and how quickly?

Task 9: Stretch Goal — Switch to GitHub Webhook

You have been using Poll SCM, which is fine for learning. In production, teams use webhooks: GitHub **pushes** a notification to Jenkins the instant a commit is made, rather than Jenkins periodically asking.

Because Jenkins is on localhost, GitHub cannot reach it directly. To complete this task you need a public tunnel.

- Install **ngrok** and expose port 8080 to the internet.
- Register the ngrok URL as a webhook in your GitHub repository settings.
- The webhook payload URL must end with `/github-webhook/`
- Switch the Jenkins build trigger from Poll SCM to **GitHub hook trigger for GITScm polling**.

- Push a commit and confirm Jenkins triggers within a few seconds — no polling delay.

This task intentionally has no hand-holding. Figure out the ngrok setup, the GitHub webhook configuration, and the Jenkins trigger setting on your own. That is the point.

4. Verify Your Pipeline

Once all tasks are complete, walk through this checklist to confirm everything is working end to end.

✓	Checkpoint
☐	The Bookshelf-API-Pipeline job exists in Jenkins.
☐	A manual 'Build Now' click completes successfully (green).
☐	Console Output shows all 4 build steps completing, including 10 tests passing.
☐	A zip artifact (bookshelf-api-build-N.zip) is visible on the build page and downloadable.
☐	Making a commit to the GitHub repository triggers an automatic build (via Poll SCM or webhook).
☐	A failing test (break something in app.py, push, revert) causes the build to go red and stop before packaging.
☐	(Stretch) A Slack message is received on both a passing and a failing build.

5. Debugging — Reading Console Output

When a build fails, the Console Output is everything. Click the build number in the Build History, then click Console Output. Here are the most common errors and where to look.

Error in Console Output	Likely Cause
`ERROR: Repository not found`	Repo URL is wrong, or the repo is private. Double-check the URL and ensure the repo is public.
`error: pathspec 'main' did not match`	Branch Specifier is wrong. The repo uses 'main' but Jenkins was configured with '*/master'.
`python: command not found`	Python is not on PATH or only available as python3. Update your shell script accordingly.
`pip: command not found`	Same as above — try pip3. Also ensure pip is installed on the Jenkins server.
`FAILED tests/test_app.py`	A test assertion failed. The console shows the diff: expected value vs. actual value.
`zip: command not found`	Install zip on the server: sudo apt install zip -y
`No artifacts captured for pattern`	The glob pattern in Archive Artifacts does not match the filename your script produced. Compare them carefully.

The Console Output is your best friend. When in doubt, read it top to bottom.